

Personalized Federated Fine-Tuning for LLMs via Data-Driven Heterogeneous Model Architectures

Yicheng Zhang
Zhejiang University
Hangzhou, Zhejiang, China
zyccs@zju.edu.cn

Zhen Qin*
Ningbo Global Innovation Center,
Zhejiang University
Ningbo, Zhejiang, China
zhenqin@zju.edu.cn

Zhaomin Wu
National University of Singapore
Singapore
zhaomin@nus.edu.sg

Jian Hou
Zhejiang Sci-Tech University
Hangzhou, Zhejiang, China
changeleap@163.com

Shuiguang Deng*
Zhejiang University
Hangzhou, Zhejiang, China
dengsg@zju.edu.cn

Abstract

Large language models (LLMs) are increasingly powering web-based applications, whose effectiveness relies on fine-tuning with large-scale instruction data. However, such data often contains valuable or sensitive information that limits its public sharing among business organizations. Federated learning (FL) enables collaborative fine-tuning of LLMs without accessing raw data. Existing approaches to federated LLM fine-tuning usually adopt a uniform model architecture, making it challenging to fit highly heterogeneous client-side data in varying domains and tasks, e.g., hospitals and financial institutions conducting federated fine-tuning may require different LLM architectures due to the distinct nature of their domains and tasks. To address this, we propose FedAMoLE, a lightweight personalized FL framework that enables data-driven heterogeneous model architectures. It features a heterogeneous mixture of low-rank adaptation (LoRA) experts module to aggregate architecturally heterogeneous models and a reverse selection-based expert assignment strategy to tailor model architectures for each client based on data distributions. Experiments across seven scenarios demonstrate that FedAMoLE improves client-side performance by an average of 5.97% over existing approaches while maintaining practical memory, communication, and computation overhead.

CCS Concepts

• **Computing methodologies** → **Distributed artificial intelligence**.

Keywords

Personalized Federated Learning, Large Language Models, Heterogeneous Model Architectures, Mixture of LoRA Experts.

ACM Reference Format:

Yicheng Zhang, Zhen Qin, Zhaomin Wu, Jian Hou, and Shuiguang Deng. 2026. Personalized Federated Fine-Tuning for LLMs via Data-Driven Heterogeneous Model Architectures. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3774904.3792147>

Resource Availability:

The source code of this paper has been made publicly available at <https://github.com/zyc140345/FedAMoLE>.

1 Introduction

Large language models (LLMs) are profoundly reshaping the contemporary web experience, serving as the core engine of many web-based applications, such as recommender systems [11], content analysis [34], and web agents [30]. The performance of these applications directly depends on the capabilities of the underlying LLMs. Instruction tuning, as a key step in shaping the exceptional capabilities of LLMs, requires large amounts of instruction-based text data, as suggested by the scaling law [18]. As publicly available data gradually depletes [44], sharing instructional data between business organizations to jointly enhance the capabilities of LLMs will effectively build a competitive advantage, since business organizations typically accumulate a large amount of instructional data while operating businesses powered by LLMs. However, the privacy information and commercial value contained in instructional data somewhat restrict the direct sharing of such data.

Federated learning (FL) [28] enables multiple business organizations to collaboratively train models without sharing raw data. Recently, it has increasingly been used to mine the value of client-side data in a privacy-preserving manner to enhance LLM capabilities [19, 35, 37]. Given the large scale of LLMs, recent efforts on federated fine-tuning mainly focus on improving system efficiency [4, 35, 55], typically following standard FL settings that produce a single globally consistent model. However, in cross-silo scenarios where different business organizations hold data from diverse domains or following various distributions, client-side data is often statistically heterogeneous (a.k.a. non-independent and identically distributed, non-IID). This makes it difficult for a single global model to generalize across all client-side data, resulting in suboptimal accuracy [41].

*Zhen Qin and Shuiguang Deng are co-corresponding authors.



This work is licensed under a Creative Commons Attribution 4.0 International License. *WWW '26, Dubai, United Arab Emirates*
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2307-0/2026/04
<https://doi.org/10.1145/3774904.3792147>

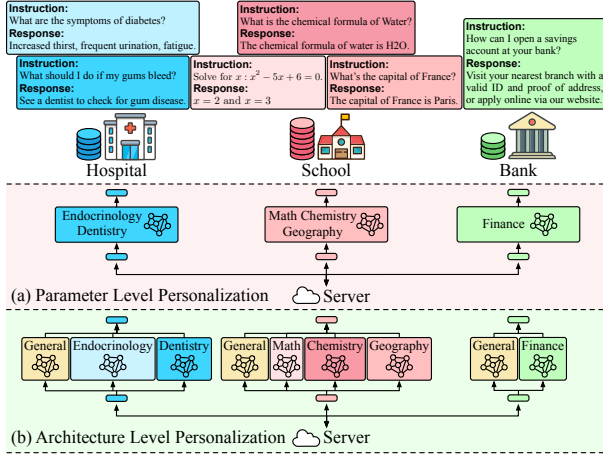


Figure 1: Scenario examples of federated LLM fine-tuning among three institutes, where each holds data that differs in domain and tasks. To better adapt to local data, personalized models with heterogeneous architectures tailored for local data distributions are usually preferred (b).

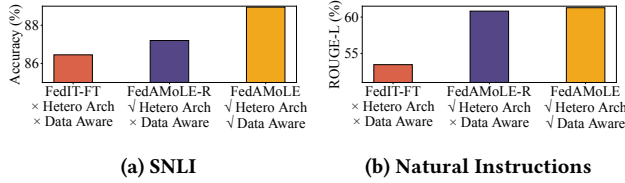


Figure 2: Effectiveness of heterogeneous and data-driven architectures. FedIT-FT and FedAMoLE-R use homogeneous and heterogeneous architectures (Hetero Arch), respectively. FedAMoLE further adopts data-driven architecture optimization. (Please refer to §5.4 for more details)

Prior FL studies indicate that enabling multiple personalized models can improve performance on client-side data [24, 36]. However, due to their dependence on external proxy data used for knowledge distillation, or otherwise high resource costs, some existing personalized FL (PFL) methods are impractical for fine-tuning LLMs (as discussed in §A.2). Recent studies have explored PFL for LLMs through distillation-free personalization [17, 33, 52]. Despite their promising results, they still face two key limitations:

- **(L1) Limited Support for Architecture-Level Personalization:** Most existing PFL methods for LLM fine-tuning focus primarily on parameter-level personalization within a fixed global architecture, as illustrated in Figures 1(a) [1, 33]. This constraint reduces the flexibility of adaptation across highly non-IID client data. As shown in Figure 2, the method that enables architectural heterogeneity through personalized submodule (expert) compositions, i.e., FedAMoLE-R, consistently outperforms another PFL approach that relies on a fixed shared architecture (FedIT-FT). Thus, it is important to explore architecture-level personalization in federated LLM fine-tuning.
- **(L2) Lack of Structural Adaptation to Local Data:** Existing FL methods supporting heterogeneous LLMs are mainly designed with manually predefined model architectures [17, 52, 56], which lack joint awareness of intra- and inter-client data distributions.

As shown in Figure 1(b), FedAMoLE (ours) optimizes personalized model architectures based on data distributions to align experts with client-side data domains, thereby improving fine-tuning accuracy (Figure 2). Thus, exploring data-aware model architectures is crucial for LLM fine-tuning in FL.

This work aims to enable the tailoring of personalized model architectures informed by client-side data distributions. To this end, we propose FedAMoLE, a PFL framework for fine-tuning personalized LLMs that supports adaptation to local data distributions through data-driven heterogeneous model architectures. It consists of two key components: 1) A **heterogeneous mixture of low-rank adaptation expert (HMoLE)** module, which employs a novel shape-invariant router to **enable aggregation of architecturally heterogeneous models (L1)**. 2) A reverse selection-based **expert assignment (RSEA)** strategy for **expert assignment**, where experts select domain-relevant clients from a global perspective, enabling data-driven architecture optimization based on client-side data distributions without direct access to local data (L2). The main contributions of this work are as follows:

- We propose FedAMoLE, a novel PFL framework for fine-tuning LLMs that enables data-driven heterogeneous model architectures. It features a module that supports heterogeneous model aggregation and a strategy for client model customization, making it uniquely capable of adapting to diverse client-side data.
- To aggregate architecturally heterogeneous models, we design HMoLE module for FedAMoLE, which employs a shape-invariant router to coordinate a heterogeneous mixture of LoRA experts.
- To tailor model structures for client-side data, we propose RSEA, a data-driven architecture optimization strategy in FedAMoLE. It adopts a novel expert-to-client selection mechanism that jointly considers intra- and inter-client data characteristics.
- We conduct extensive experiments across seven non-IID FL scenarios, showing that FedAMoLE improves task accuracy by an average of 5.97% over the strongest baseline.

2 Related Work

This section briefly overviews existing studies highly related to this work. For detailed discussions, please refer to Appendix A.

Personalized Federated LLM Fine-Tuning Currently, most FL methods for LLM fine-tuning adopt Parameter-Efficient Fine-Tuning (PEFT) techniques such as Low-Rank Adaptation (LoRA) [37, 50, 55] and prompt tuning [4] to reduce resource costs. These methods typically yield a single global model that fails to adapt to heterogeneous client-side data. Recent studies explore personalization via dual adapters [33], LoRA masking [56], and mixture of Feed-Forward Networks (FFN) [29], but they either (1) support only parameter-level personalization [33], which may struggle with non-IID data; (2) rely on manually defined model architectures [56], which may not align well with client data; or (3) incur significant communication costs from transmitting large-scale parameters [29].

Federated LLM Fine-Tuning with Mixture of Experts The Mixture-of-Experts (MoE) architecture [9, 16] offers a promising basis for PFL by enabling expert-group customization. Recent MoE-based PFL methods adopt Mixture of LoRA Experts (MoLE) to improve fine-tuning efficiency [46], but only support parameter personalization. Efforts on architectural heterogeneity either rely on dense FFN

experts [29], which incur high communication costs and incompatibility with non-MoE LLMs like LLaMA [43], or adopt manually defined model architectures [8], limiting adaptation to local data. In contrast, our method enables data-driven architectures, enhancing personalization while maintaining low communication cost.

3 Problem Formulation

We provide a formal definition of PFL for LLM fine-tuning. In FL, C clients collaboratively solve:

$$\min_{\theta} \mathcal{L} = \frac{1}{C} \sum_{i=1}^C \mathbb{E}_{\mathbf{X} \sim \mathcal{D}_i} \mathcal{L}_i(\mathbf{X}|\theta), \quad (1)$$

aiming at optimizing a global model θ by minimizing the expected loss $\mathcal{L}_i$ over local data distribution $\mathcal{D}_i$ for each client i . To address data heterogeneity across clients, PFL trains a personalized model θ_i for each client i , modifying the objective to:

$$\min_{\theta_1, \theta_2, \dots, \theta_C} \mathcal{L} = \frac{1}{C} \sum_{i=1}^C \mathbb{E}_{\mathbf{X} \sim \mathcal{D}_i} \mathcal{L}_i(\mathbf{X}|\theta_i). \quad (2)$$

For LLM fine-tuning, $\mathbf{X} = [\mathbf{X}^{\text{instr}}, \mathbf{X}^{\text{resp}}]$ denotes a text sequence, where $\mathbf{X}^{\text{instr}}$ is the instruction and $\mathbf{X}^{\text{resp}}$ is the response. $\mathcal{L}_i$ is the negative log-likelihood (NLL) loss defined as:

$$\mathcal{L}_i(\mathbf{X} | \theta_i) = -\frac{1}{T} \sum_{t=1}^T \log[p(\mathbf{X}_t^{\text{resp}} | \mathbf{X}_t^{\text{instr}}, \mathbf{X}_{<t}^{\text{resp}}, \theta_i)], \quad (3)$$

where T is the response sequence length. In typical PFL, θ_i uses a uniform structure with varying parameters, while FedAMoLE enables architectural heterogeneity for greater personalization.

4 Approach

4.1 Framework Overview

Figure 3 overviews the components and processes of FedAMoLE. Each client has a local model with a frozen LLM backbone and trainable *HMoLE modules*. Clients inject distinct HMoLE modules into corresponding linear layers, which allows architectural heterogeneity even with a common backbone. The architecture of each HMoLE module is determined in a data-driven manner: for each activated linear layer, the server maintains a global HMoLE module comprising a *shared expert* to capture general knowledge, a pool of *domain experts* to learn specialized patterns, and a *token projection* to route input tokens. The shared expert and token projection are broadcast to all clients, while client-specific subsets of domain experts are assigned with *RSEA strategy* based on domain similarity, to form personalized HMoLE modules. This design tailors personalized model architectures to both intra- (via domain experts) and inter-client (via shared experts) data distributions, and maximizes architectural flexibility through module-level expert assignment.

In FedAMoLE, each FL round consists of six steps. First, the server distributes global parameters to all clients, including shared experts, domain experts, and token projections (Figure 3, ①). Next, each client constructs HMoLE modules using these parameters and injects them into the shared LLM backbone to form its personalized model, which is then fine-tuned on its local training set (Figure 3, ②). Then, each client computes embeddings for its domain experts and local data using a randomly sampled subset of its training set,

termed *embedding set* (Figure 3, ③), and returns the fine-tuned parameters along with these embeddings to the server (Figure 3, ④). Finally, the server aggregates the received parameters and embeddings (Figure 3, ⑤), and applies the RSEA strategy to determine the model architecture for each client in the next round (Figure 3, ⑥).

4.2 Heterogeneous MoLE Module

While vanilla MoLE enables personalized architectures in a light-weight manner, it poses aggregation challenges in FL due to expert-number-dependent router shapes (see §B). To address this, we propose the HMoLE module, which decouples router shape from local expert configuration, thereby enabling lightweight model heterogeneity with FedAvg compatibility.

The routing process of HMoLE is shown in Figure 4. Let $h_{m,i}(\mathbf{x}) \in \mathbb{R}^d$ denote the hidden state of input token $\mathbf{x}$ in module m of client i . It is projected into a low-dimensional subspace by matrix $\mathbf{A}$ of each domain expert j , generating the corresponding expert embedding $h_{m,i,j}^e(\mathbf{x}) \in \mathbb{R}^r$. Also, $h_{m,i}(\mathbf{x})$ is projected by the token projection matrix $\mathbf{W}_{m,i}^t \in \mathbb{R}^{r \times d}$ to obtain the token embedding $h_{m,i}^t(\mathbf{x}) \in \mathbb{R}^r$ in this subspace. Next, the scaled dot product between the token embedding and each expert embedding is computed, followed by softmax normalization to obtain the weighting coefficient $p_{m,i,j}(\mathbf{x})$ for each domain expert j . Finally, k^e domain experts with the highest coefficients (where $k^e = 2$ in Figure 4) are selected to extract features $\phi_{m,i,j}(\mathbf{x})$. These features are averaged based on the corresponding coefficients, and combined with the feature from the shared expert $\phi_{m,i}^s(\mathbf{x})$ and the pre-trained feature $\phi_m^p(\mathbf{x})$ to get the final output $y_{m,i}(\mathbf{x})$. This process can be formalized as

$$h_{m,i,j}^e(\mathbf{x}) = \mathbf{A}_{m,i,j} h_{m,i}(\mathbf{x}) \quad (4)$$

$$h_{m,i}^t(\mathbf{x}) = \mathbf{W}_{m,i}^t h_{m,i}(\mathbf{x}) \quad (5)$$

$$p_{m,i,j}(\mathbf{x}) = \text{softmax} \left[\frac{h_{m,i}^t(\mathbf{x})^T h_{m,i,j}^e(\mathbf{x})}{\sqrt{d}} \right] \quad (6)$$

$$\mathcal{T}_{m,i}(\mathbf{x}) = \text{top-}k^e(p_{m,i,j}(\mathbf{x})) \quad (7)$$

$$\phi_m^p(\mathbf{x}) = \mathbf{W}_m h_{m,i}(\mathbf{x}) \quad (8)$$

$$\phi_{m,i}^s(\mathbf{x}) = \mathbf{B}_{m,i}^s \mathbf{A}_{m,i}^s h_{m,i}(\mathbf{x}) \quad (9)$$

$$\phi_{m,i,j}(\mathbf{x}) = \mathbf{B}_{m,i,j} \mathbf{A}_{m,i,j} h_{m,i}(\mathbf{x}) \quad (10)$$

$$y_{m,i}(\mathbf{x}) = \phi_m^p(\mathbf{x}) + \phi_{m,i}^s(\mathbf{x}) + \sum_{j \in \mathcal{T}_{m,i}(\mathbf{x})} p_{m,i,j}(\mathbf{x}) \cdot \phi_{m,i,j}(\mathbf{x}), \quad (11)$$

where $\mathbf{W}_m$ is the pre-trained weight, $\mathbf{A}_{m,i,j}$ and $\mathbf{B}_{m,i,j}$ are the parameters of domain expert j , $\mathbf{A}_{m,i}^s$ and $\mathbf{B}_{m,i}^s$ are those of the shared expert, and k^e is the number of domain experts selected per token.

4.3 Local Fine-Tuning and Aggregation

In each round, client i freezes the pre-trained backbone and trains the parameters of its HMoLE modules $\mathcal{M}$, i.e., $\theta_i = \bigcup_{m \in \mathcal{M}} \theta_{m,i}$, where $\theta_{m,i} = \{\mathbf{A}_{m,i}^s, \mathbf{B}_{m,i}^s, \mathbf{W}_{m,i}^t\} \cup \{\mathbf{A}_{m,i,j}, \mathbf{B}_{m,i,j}\}_{j \in \mathcal{E}_{m,i}}$ is the trainable parameters of module m on client i , and $\mathcal{E}_{m,i}$ is the set of domain experts assigned to this module. In each iteration, client i samples a mini-batch $\mathcal{B}$ from its training set $\mathcal{D}_i^{\text{train}}$ and computes the gradient of its parameters θ_i w.r.t. the loss function $\mathcal{L}_i$ (detailed in §C). The parameters are then updated by $\theta_i^{t+1} = \theta_i^t - \eta \nabla_{\theta_i^t} \mathcal{L}_i(\mathcal{B}|\theta_i^t)$, where η is the learning rate.

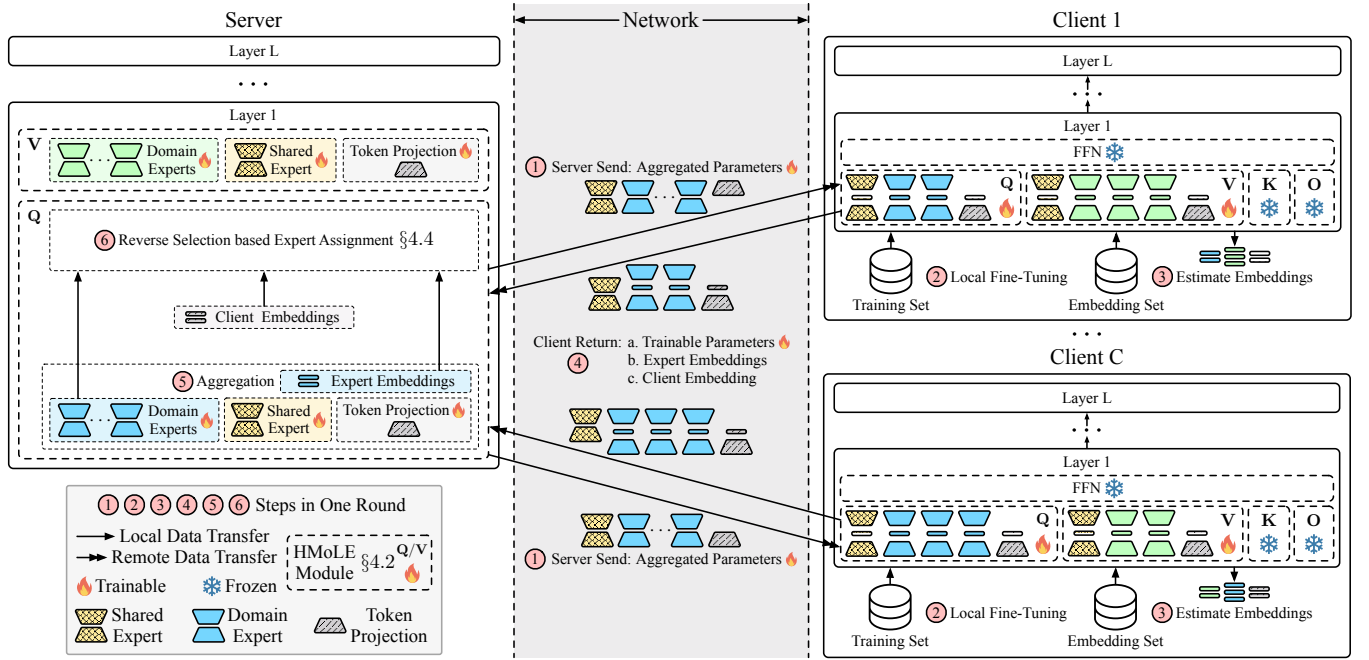


Figure 3: Overview of FedAMoLE in a single round. Each client has a transformer-based local model with L decoder layers, where each layer includes a self-attention block (with parameters Q , K , V , and O) and an FFN block. Trainable HMOLe modules (see Figure 4) are injected into Q and V for fine-tuning, while K , O , and FFN remain frozen. Step ⑥ denotes the RSEA strategy (see Figure 5). Components of the same type (e.g., all shared experts) are shown with an identical color and texture.

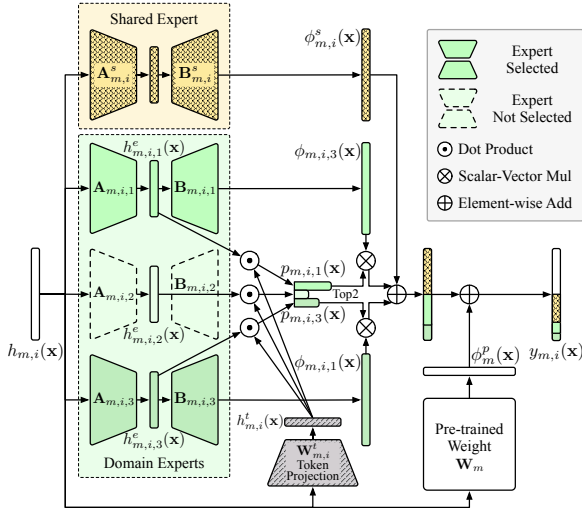


Figure 4: Routing of HMOLe module m at client i .

After fine-tuning, each HMOLe module m performs federated aggregation as

$$X_{m,j} = \frac{1}{|C_{m,j}|} \sum_{i \in C_{m,j}} X_{m,i,j}, \quad Y_m = \frac{1}{C} \sum_{i=1}^C Y_{m,i}, \quad (12)$$

where $C_{m,j}$ denotes the set of clients participating in fine-tuning domain expert j within module m in this round, X is applied to both A and B , Y is applied to both A^s , B^s and W^t . Owing to the small scale of HMOLe modules (around 1% of the total parameters),

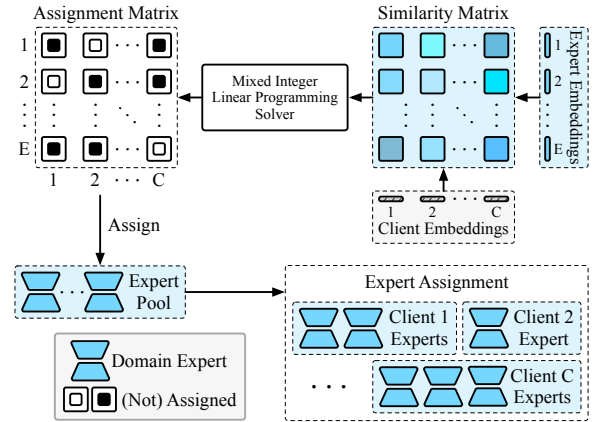


Figure 5: RSEA process. E and C denote the number of domain experts and clients, respectively.

memory overhead of local fine-tuning and the communication cost of aggregation remain minimal (Tables 2 and 3).

4.4 Expert Assignment with Reverse Selection

To enable data-aware model architectures in MoE-based FL, traditional approaches [7] adopt a forward selection strategy: each client first downloads all experts from the server and evaluates their relevance to its local data using a predefined metric. Then, each client selects the k most relevant experts to form its local model. However, this strategy (1) enforces an identical model structure, as all clients select the same number of experts; (2) may lead to

suboptimal expert assignment, as it considers only local data and ignores the collaborative nature of expert training; (3) risks privacy leakage, as clients may infer peers' data characteristics by accessing all experts; and (4) incurs high communication cost from downloading the full expert pool.

To address these limitations, we propose a **reverse selection-based expert assignment (RSEA)** strategy (see Figures 3 and 5): each client first uploads embeddings of its local data and assigned experts. On the server side, *each expert then selects the k most relevant clients* based on an embedding similarity metric. Finally, each client downloads only the experts that have selected it to construct the local model. Compared to forward selection, RSEA (1) enables model heterogeneity by allowing each client to receive a different number of experts; (2) approximates optimal expert assignment by considering data characteristics from all clients; (3) enhances privacy by restricting access to only relevant experts; and (4) reduces communication costs by limiting downloads to a subset of experts.

RSEA strategy requires a metric to quantify the relevance between client data and expert domains. Thus, we define the embedding of client i on module m as the element-wise mean of token embeddings in m for all tokens in the embedding set $\mathcal{D}_i^{\text{emb}}$ sampled from the training set $\mathcal{D}_i^{\text{train}}$:

$$\bar{\mathbf{h}}_{m,i}^t = \frac{1}{|\mathcal{D}_i^{\text{emb}}|} \sum_{\mathbf{x} \in \mathcal{D}_i^{\text{emb}}} h_{m,i}^t(\mathbf{x}). \quad (13)$$

Note that the sampling of $\mathcal{D}_i^{\text{emb}}$ is random to ensure its data distribution aligns with $\mathcal{D}_i^{\text{train}}$, thereby better reflecting the local data characteristics. Similarly, we define the embedding of domain expert j in module m as the element-wise mean of j 's embeddings across clients $C_{m,j}$ that fine-tune j in the current round (14), where $\bar{\mathbf{h}}_{m,i,j}^e$ is the embedding of j on i , i.e., the element-wise mean of j 's embeddings in m for all tokens in $\mathcal{D}_i^{\text{emb}}$ (15).

$$\bar{\mathbf{h}}_{m,j}^e = \frac{1}{|C_{m,j}|} \sum_{i \in C_{m,j}} \bar{\mathbf{h}}_{m,i,j}^e \quad (14)$$

$$\bar{\mathbf{h}}_{m,i,j}^e = \frac{1}{|\mathcal{D}_i^{\text{emb}}|} \sum_{\mathbf{x} \in \mathcal{D}_i^{\text{emb}}} h_{m,i,j}^e(\mathbf{x}) \quad (15)$$

The metric $s_{m,i,j}$, measuring the relevance between client i 's data and expert j 's domain within module m , is then defined as the scaled dot product of their embeddings (16), which aligns with the HMoLE training objective (4–6). A higher $s_{m,i,j}$ signifies a stronger relevance, indicating a greater likelihood of expert j being selected by client i 's tokens in module m . This design establishes a linkage between experts and the hidden domains in client data, which gradually stabilizes during training and ultimately aligns each expert with the most relevant clients (see §5.5).

$$s_{m,i,j} = \frac{(\bar{\mathbf{h}}_{m,i}^t)^T \cdot \bar{\mathbf{h}}_{m,j}^e}{\sqrt{d}} \quad (16)$$

With this metric, each domain expert j in module m selects k^c clients with the highest relevance:

$$C_{m,j} = \text{top-}k^c(\text{softmax}(s_{m,i,j})). \quad (17)$$

Note that we avoid using a threshold-based approach here, as (1) a continuous threshold introduces a much larger search space

than discrete client counts, making hyperparameter tuning more difficult, and (2) thresholding may waste model capacity, e.g., if the threshold is 0.4 and the relevance scores of an expert to three clients are all 1/3, the expert would not be assigned to any client.

However, the reverse selection strategy (17) may lead to imbalanced expert assignments, where some clients are selected by no experts while others are selected by all. To constrain the minimum number of experts per module (k^e) and limit expert heterogeneity (up to b experts per module per client), we formulate expert assignment within each module as the following optimization problem:

Definition 4.1. (Domain Expert Assignment Problem)

$$\begin{aligned} & \max_{\mathbf{D}_m} \langle \mathbf{P}_m, \mathbf{D}_m \rangle \\ \text{s.t. } & \forall i : k^e \leq \sum_j d_{m,i,j} \leq b; \forall j : \sum_i d_{m,i,j} = k^c; \forall i, j : d_{m,i,j} \in \{0, 1\}. \end{aligned}$$

Here, $\mathbf{P}_m = \{p_{m,i,j}\}$ is the probability matrix for selection, where $p_{m,i,j} = e^{s_{m,i,j}} / \sum_i e^{s_{m,i,j}}$ denotes the probability that domain expert j in module m selects client i . $\mathbf{D}_m = \{d_{m,i,j}\}$ is the binary assignment matrix, with $d_{m,i,j} = 1$ if expert j is assigned to client i , and 0 otherwise. The objective maximizes the inner product $\langle \mathbf{P}_m, \mathbf{D}_m \rangle$, aligning expert assignments with selection preferences. The first constraint bounds the number of experts assigned to each client between k^e and b for balance, while the second constraint enforces that each expert selects exactly k^c clients, as in (17).

Problem 4.1 can be reduced to a mixed integer linear programming (MILP) problem, which is $\mathcal{NP}$ -hard. However, for domain expert assignment, the problem size is typically small (a few hundred decision variables) and can be quickly solved using solvers such as Gurobi [13] or SCIP [2].

4.5 Enhancement on Privacy

RSEA strategy requires each client i to upload its expert embeddings $\{\bar{\mathbf{h}}_{m,i,j}^e\}_{j \in \mathcal{E}_{m,i}}$ and data embedding $\bar{\mathbf{h}}_{m,i}^t$. While these embeddings do not directly reveal raw data, techniques such as differential privacy (DP) [48] can be employed to provide stronger and more comprehensive privacy protection. For example, to privatize $\bar{\mathbf{h}}_{m,i,j}^e$, we can adopt the DP-based Split-N-Denoise method [26], which adds Laplace noise $\mathbf{z} \sim c \cdot \exp(-\eta \cot \|\mathbf{z}\|_2)$ (with constants c and η) to obtain $M(\bar{\mathbf{h}}_{m,i,j}^e) = \bar{\mathbf{h}}_{m,i,j}^e + \mathbf{z}$. The result is then l_2 -clipped as $M'(\bar{\mathbf{h}}_{m,i,j}^e) = M(\bar{\mathbf{h}}_{m,i,j}^e) \cdot \min(1, C/\|M(\bar{\mathbf{h}}_{m,i,j}^e)\|_2)$, where C is the upper bound of $\|\bar{\mathbf{h}}_{m,i,j}^e\|_2$. Finally, $M'(\bar{\mathbf{h}}_{m,i,j}^e)$ satisfies ηd_X -privacy and can be safely uploaded. We further show in experiments that FedAMoLE maintains strong performance with DP (see §D.6).

5 Experiments

The experiments aim at demonstrating: 1) FedAMoLE achieves better accuracy on non-IID data (§5.2), 2) FedAMoLE has lower resource overhead than MoE-based approaches (§5.3), and 3) the components proposed in this work contribute positively (§5.4–5.5).

5.1 Experimental Setup

Baselines. We compare FedAMoLE to 8 federated LLM fine-tuning methods, encompassing both non-personalized and personalized ones. Non-personalized ones include FedIT [55], FedPrompt, and

Table 1: Comparison of fine-tuning accuracy (MTAL) across seven scenarios. Each cell reports the mean and standard deviation of MTAL calculated over three random seeds. Each column represents the results under a specific data heterogeneity scenario, where the best-performing approach is highlighted in bold, the second-best method is underlined, “gains” denote the relative MTAL improvement of FedAMoLE over the best baseline, and “-” indicates failure to converge in the official implementation.

Approach		BigBench 1 domain/client	Hybrid 1 task/client	NI 1 task/client	Dolly-15K			SNLI $\alpha = 1.0$	Average
Vanilla	FedIT	43.84±1.48	59.26±0.72	54.09±0.23	28.69±1.73	28.67±1.28	<u>27.31±0.55</u>	83.06±0.63	46.42
	FedPrompt	26.80±1.95	39.93±1.51	35.45±1.08	26.01±2.16	26.44±1.22	24.57±0.78	66.10±2.78	35.04
	FedPTuning	42.45±0.79	50.92±11.78	36.10±18.36	24.11±3.14	27.57±0.79	26.16±1.12	76.83±6.14	40.59
Personalized	FedIT-FT	47.67±1.18	60.17±0.90	53.34±0.26	<u>28.80±0.16</u>	<u>29.13±1.18</u>	27.17±0.40	<u>86.67±0.58</u>	47.56
	FedPrompt-FT	26.68±1.05	40.87±2.06	47.70±0.67	26.76±0.57	25.91±0.98	24.63±1.17	75.75±1.46	38.33
	FedPTuning-FT	43.60±1.36	52.39±12.56	38.97±20.72	27.28±1.28	28.05±1.42	25.89±0.43	86.34±3.20	43.22
	CoMiGS	29.40±0.80	35.25±1.44	25.26±0.67	-	-	-	62.09±2.40	38.00
	FDLoRA	<u>48.23±1.85</u>	<u>60.31±0.95</u>	<u>57.04±1.90</u>	27.75±0.17	26.67±0.66	25.79±1.99	86.06±1.32	47.41
	FedAMoLE	52.56±1.43	63.60±0.30	60.04±1.09	29.87±1.40	29.72±1.43	28.28±0.84	88.78±0.38	50.41
Gains (%)		8.97	5.46	5.25	3.72	2.02	3.55	2.43	5.97 ($p < 0.01$)

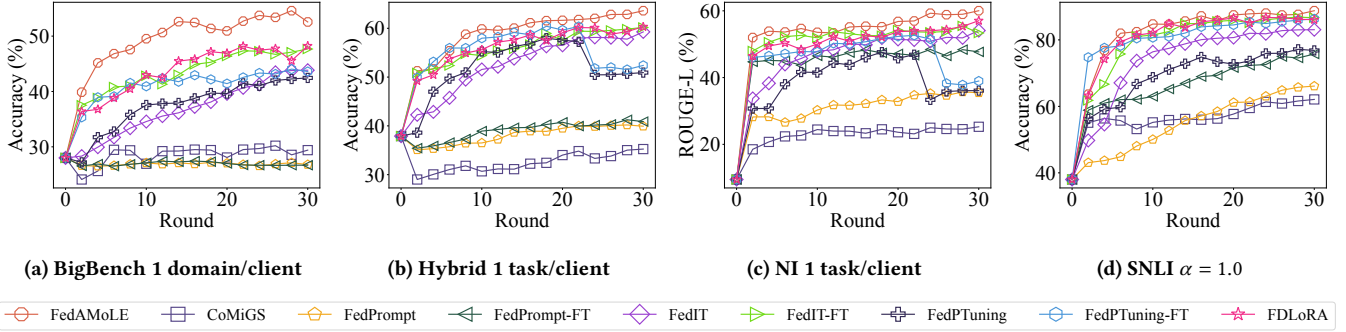


Figure 6: Trends of MTA among comparison methods, averaged over three runs with different seeds.

FedPTuning [19], which adopt FedAvg [28] with LoRA [15], Prompt Tuning [20], and P-Tuning [25] respectively. Personalized ones include FedIT-FT, FedPrompt-FT, FedPTuning-FT (each tunes its trainable modules after aggregation [54]), FDLORA [33] which employs shared and personalized adapters combined via few-shot black-box optimization, and CoMiGS [8] which uses manually defined personalized MoLE structures where clients alternately train local routers and experts, aggregating only the shared expert.

FL Settings. Unless otherwise stated, we construct a 10-client cross-silo FL scenario. All experiments use LLaMA-3.2-1B¹ as the foundation model and run for 30 rounds, with each client performing 200 local updates per round.

Datasets. To comprehensively evaluate FL methods in realistic heterogeneous data scenarios, we construct three kinds of non-IID data heterogeneity. Aligning with scenarios of collaboration across organizations from distinct domains, we first introduce heterogeneity from domain skew by assigning each of the 10 clients a task from a distinct domain in BigBench [40]. For scenarios of collaboration among organizations holding different task types within the same or similar domains, we then create heterogeneity from task skew, where each client is assigned a unique task from Natural Instructions (NI) [47] or our custom Hybrid dataset (contains 10 subsets,

detailed in Appendix D.1). Finally, we simulate heterogeneity via label skew following prior studies [19, 36] by partitioning SNLI [3] and Dolly-15K [6] among clients using a Dirichlet distribution with α values of 0.1, 1.0, and 100.0.

Evaluation. Following prior studies on personalized federated fine-tuning [33, 52], we construct an in-domain/task test set for each client and adopt **mean test accuracy at the last round (MTAL)** as the evaluation metric, defined as:

$$\text{MTAL} = \text{MTA}[-1]. \quad (18)$$

Here, $\text{MTA}[t] = \frac{1}{C} \sum_{i=1}^C \text{acc}_i[t]$, where $\text{acc}_i[t]$ evaluates the performance of client i 's local model at round t . For classification tasks, acc denotes accuracy, while for natural language generation tasks, acc corresponds to ROUGE-L [23] following prior studies on federated LLM tuning [19, 35].

5.2 Comparison of Accuracy

From Table 1, FedAMoLE achieves the best results across all seven scenarios. Its advantages are most pronounced on the highly heterogeneous datasets—BigBench, Hybrid, and NI—where each client's data belongs to a unique domain or task. On these, FedAMoLE outperforms the strongest baseline, FDLORA, by substantial margins of 8.97% on BigBench, 5.46% on Hybrid, and 5.25% on NI. In contrast, CoMiGS, another MoLE-based method, struggles on these

¹Adopt checkpoint from <https://huggingface.co/meta-llama/Llama-3.2-1B>.

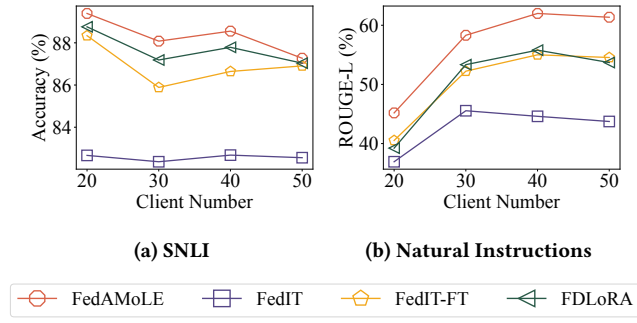


Figure 7: MTAL with different numbers of clients.

complex tasks, highlighting the superiority of FedAMoLE’s design. This stems from three key advantages: 1) its data-driven architecture better adapts to local data than CoMiGS’s manual setup; 2) its aggregatable router provides superior generalization over a local-only router; and 3) its domain experts are collaboratively trained, enhancing performance beyond single-client training. Furthermore, FedAMoLE’s effectiveness extends to label-skewed scenarios, improving accuracy by 2.43% on SNLI ($\alpha = 1.0$) over FedIT-FT while also consistently achieving the best performance across all Dolly-15K settings. This consistent gain is due to FedAMoLE’s data-aware, module-level adaptive expert assignment strategy, which builds personalized models tailored to each client’s local data, enhancing performance on heterogeneous data (§5.5).

Additionally, as shown in Figure 6, FedAMoLE converges quickly across four typical non-IID scenarios and consistently outperforms other approaches during training. This indicates that even if training is stopped early for efficiency, FedAMoLE still achieves better results, demonstrating its practicality. This advantage comes from FedAMoLE’s ability to dynamically adjust expert assignments, allowing it to respond promptly to changes in relevance between experts and client data as training progresses. Furthermore, Figure 6 shows a clear performance hierarchy based on the degree of personalization: moving from vanilla FL to parameter-level personalization (via post-aggregation fine-tuning) yields significant gains, while FedAMoLE’s architecture-level personalization achieves the best results across all four non-IID scenarios. This underscores that a greater degree of model personalization is key to superior performance on heterogeneous data.

Figure 7 shows the effect of client count. Notably, FedAMoLE consistently outperforms the strongest baselines under both label- and task-skewed settings, demonstrating its strong scalability.

Further accuracy comparisons on varied *LLM sizes*, *backbones*, and *datasets* can be found in §D.2.

5.3 Comparison of Overhead

Table 2 presents the system efficiency of various PFL baselines and FedAMoLE. The RSEA process latency is included in FedAMoLE’s aggregation time, while the black-box optimization for FDLORA is part of its testing latency. We observe that despite employing multiple experts, FedAMoLE’s memory usage remains comparable to single LoRA-based baselines and is nearly unaffected by the total number of experts, since each expert is a lightweight LoRA adapter with negligible parameters compared to the pre-trained

Table 2: Comparison of per-client system overhead in one FL round on the SNLI dataset using the LLaMA-3.2-1B backbone. “Mem” is the peak client-side GPU memory usage. “Up” and “Down” are the data sizes for client-to-server and server-to-client transfers, respectively. “Train,” “Aggr,” and “Test” are the latencies for local training, server-side aggregation, and local testing. FedAMoLE (X) denotes a total of X experts per module; FedAMoLE* indicates an optimized parallel implementation; “+” denotes the overhead relative to FedIT-FT.

Approaches	Mem/GB	Up/MB	Down/MB	Train/s	Aggr/s	Test/s
FedIT-FT	2.78	1.64	1.64	4.85	0.02	2.37
FedPrompt-FT	2.83	0.29	0.29	4.47	0.00	3.20
FedPTuning-FT	2.84	0.16	0.16	4.19	0.00	2.99
CoMiGS	8.00	21.55	21.55	52.67	0.04	19.21
FDLoRA	2.78	1.64	1.64	4.14	0.03	2.60
FedAMoLE (15)	2.79 ^{+0.01}	7.59 ^{+6.0}	7.59 ^{+6.0}	13.25 ^{+8.4}	4.97 ^{+5.0}	4.92 ^{+2.6}
FedAMoLE (20)	2.79 ^{+0.01}	9.24 ^{+7.6}	9.23 ^{+7.6}	14.44 ^{+9.6}	5.20 ^{+5.2}	5.20 ^{+2.8}
FedAMoLE (25)	2.81 ^{+0.03}	10.88 ^{+9.2}	10.87 ^{+9.2}	15.57 ^{+10.7}	5.49 ^{+5.5}	5.63 ^{+3.3}
FedAMoLE (30)	2.80 ^{+0.02}	12.52 ^{+10.9}	12.51 ^{+10.9}	16.49 ^{+11.6}	5.85 ^{+5.8}	5.72 ^{+3.4}
FedAMoLE* (30)	3.55 ^{+0.77}	12.52 ^{+10.9}	12.51 ^{+10.9}	12.40 ^{+7.6}	5.10 ^{+5.1}	2.82 ^{+0.5}

Table 3: System overhead versus FedMoE. Setup and notation follow Table 2.

Metric	FedMoE	FedAMoLE
Mem/GB	13.89	7.99
Up/MB	1850	25.66
Down/MB	1850	25.63

Table 4: RSEA expert assignment time (s) on a single Intel Core i5-13400F CPU.

Client	Expert			
	10	50	100	
30	0.57	2.17	4.37	
50	0.90	3.45	6.87	
100	1.63	7.62	15.28	

model. Even with 30 experts per module, the per-client data transfer is only 12.52 MB. With an optimized implementation, FedAMoLE achieves substantial reductions in both memory and computational overhead compared to another MoE-based approach, CoMiGS.

FedAMoLE also maintains low per-round training (16.49 s) and inference (5.72 s) latencies even when fine-tuning a frozen 1B-parameter LLM with 30 experts. The increased latency compared to non-MoE baselines is attributed to the additional overhead from RSEA strategy and the default memory-efficient implementation of the HMoE module, which processes domain experts sequentially. However, this latency can be mitigated by parallelizing these computations at the cost of a minor increase in memory, as demonstrated by FedAMoLE*. We expect such overheads to diminish further with future MoE hardware and software optimizations.

To further demonstrate efficiency, we compare FedAMoLE with MoE-based methods that utilize dense experts. Since FedMoE [29] is not open-source, we cannot directly measure its performance on our LLaMA-based setup. To ensure a fair comparison, we aligned FedAMoLE with its original experimental setting (a Switch Transformer backbone on the AG News dataset) and compared our results with the data reported in their paper. As shown in Table 3, FedAMoLE leverages LoRA’s parameter efficiency to compress the per-round communication cost to just 25.66 MB (approximately 1% of FedMoE’s reported GB-level volume) and reduces memory

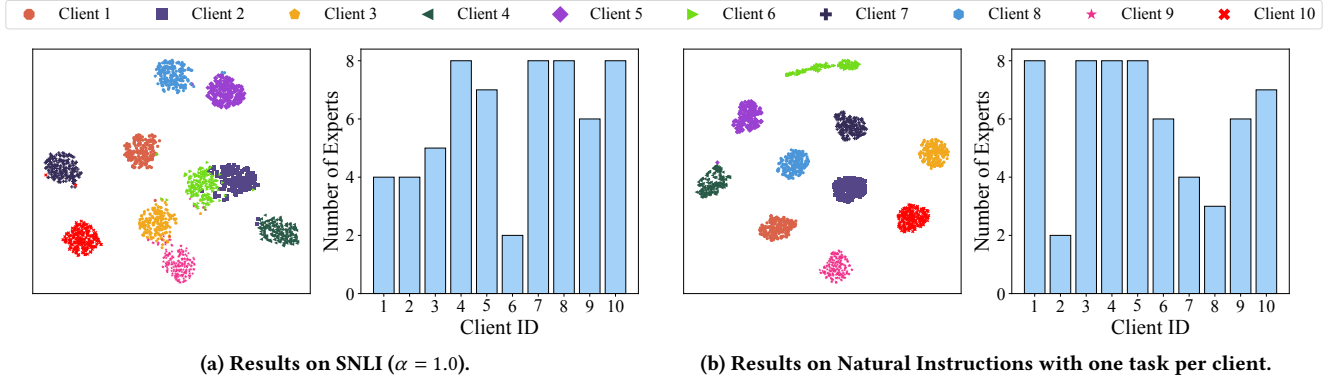


Figure 8: Visualization of the relevance between client data and domain experts in a HMoLE module after one round of FL. Each point in the scatter plot represents the t-SNE reduced relevance vector between a text sequence and all domain experts, with 200 points per client. The bar chart illustrates the number of domain experts assigned to each client in the next round.

Table 5: Ablation study conducted on SNLI and NI.

Dataset	FedIT	FedAMoLE				
	+FT	-H	-S	-R	Random	
SNLI	86.45	86.90	87.50	87.20	88.05	88.95
NI	53.45	59.57	56.04	60.82	52.92	61.29

usage by nearly half, making federated LLM fine-tuning practical in real-world scenarios.

Finally, we evaluate the scalability of RSEA strategy. From Table 4, the primary overhead of RSEA—expert assignment—takes only 15s even with 100 clients and 100 experts—a typical upper bound for cross-silo scenarios [14]. Considering that the per-round time is dominated by local training and communication, our approach is highly practical for typical cross-silo FL deployments while the underlying MILP problem is $\mathcal{NP}$ -hard in the worst case.

5.4 Ablation Study

To assess the contributions of the proposed HMoLE module, RSEA strategy, and shared expert, we design four ablation variants: (1) FedAMoLE-H, which replaces HMoLE with vanilla MoLE (21)—each router uses a fixed output dimension equal to the total number of assignable experts to ensure aggregation compatibility, with top- k^e routing restricted to assigned experts; (2) FedAMoLE-S, where the shared expert is ablated, and only domain expert collaboration is used; (3) FedAMoLE-R, where the RSEA strategy is removed and experts are manually assigned in the first round, remaining fixed thereafter; (4) FedAMoLE (Random), where the RSEA strategy is removed and experts are randomly assigned in each round.

As presented in Table 5, FedAMoLE-R significantly outperforms FedIT-FT in accuracy, demonstrating the benefits of heterogeneous models. FedAMoLE achieves superior accuracy over FedAMoLE-R, highlighting the advantages of data-driven model architectures. The performance degradation of both FedAMoLE-H and FedAMoLE (Random), relative to FedAMoLE, underscores the importance of the HMoLE module and the RSEA strategy. Moreover, the inferior performance of FedAMoLE-S compared to FedAMoLE emphasizes the necessity of shared experts for general knowledge sharing, rather than relying solely on domain experts.

5.5 Discussion on Expert Assignment

Figure 8 shows that RSEA adaptively allocates domain experts based on data distribution. In the scatter plot, each point represents the t-SNE-reduced relevance vector between a text sequence and all domain experts. This vector is calculated via (13-16), where $\mathcal{D}_i^{\text{emb}}$ in (13) contains only one sample. Points clustered near the origin signify uniform relevance, indicating an alignment with the global distribution that requires fewer domain experts. Conversely, points distant from the origin show a strong affinity for specific experts, indicating more complex, specialized data that necessitates a greater model capacity provided by more domain experts. This pattern is clearly reflected in FedAMoLE’s expert assignments. For example, in Figure 8b, the data points for clients 2, 7, and 8 are concentrated near the origin, indicating their local data aligns with the global distribution. As a result, the corresponding bar chart confirms that these clients are assigned fewer experts. Please refer to §D.4 for further discussion on the stability of expert assignment.

6 Conclusion

This work introduces FedAMoLE, a novel PFL framework for LLM fine-tuning, characterized by a heterogeneous MoLE module to enable architectural model heterogeneity, and a reverse selection-based expert assignment strategy to optimize model architectures in a data-driven manner. It enables architecture-level model personalization across clients with low communication overhead. Extensive experiments on seven scenarios demonstrate that FedAMoLE achieves superior accuracy over existing methods in scenarios with non-IID data, highlighting the potential of data-driven optimization for model architectures in LLM fine-tuning with FL.

Acknowledgments

This work was supported in part by the National Science Foundation of China under Grant 62125206, in part by the National Key Research and Development Program of China under Grant 2022YFB4500100, and in part by the Zhejiang Provincial Natural Science Foundation of China under Grant LD24F020014, and in part by the Zhejiang Pioneer (Jianbing) Project (2026C01024).

References

- [1] Abdulla Jasem Almansoori, Samuel Horváth, and Martin Takáč. 2024. Collaborative and efficient personalization with mixtures of adapters. *arXiv preprint arXiv:2410.03497* (2024).
- [2] Suresh Bolusani, Mathieu Besançon, et al. 2024. *The SCIP optimization suite 9.0*. Technical Report. Optimization Online.
- [3] Samuel Bowman, Gabor Angeli, Christopher Potts, and Christopher D Manning. 2015. A large annotated corpus for learning natural language inference. In *Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing*. 632–642.
- [4] Tianshi Che, Ji Liu, Yang Zhou, Jiaxiang Ren, Jiwen Zhou, Victor Sheng, Huaiyu Dai, and Dejing Dou. 2023. Federated Learning of Large Language Models with Parameter-Efficient Prompt Tuning and Adaptive Optimization. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. 7871–7888.
- [5] Liam Collins, Hamed Hassani, Aryan Mokhtari, and Sanjay Shakkottai. 2021. Exploiting shared representations for personalized federated learning. In *Proceedings of the 38th International Conference on Machine Learning (ICML)*. 2089–2099.
- [6] Mike Conover, Matt Hayes, et al. 2023. Free Dolly: Introducing the World's First Truly Open Instruction-Tuned LLM. Databricks Blog.
- [7] Chen Dun, Mirian Hipolito Garcia, Guoqing Zheng, Ahmed Hassan Awadallah, Robert Sim, Anastasios Kyrillidis, and Dimitrios Dimitriadis. 2023. Fedjets: Efficient just-in-time personalization with federated mixture of experts. *arXiv preprint arXiv:2306.08586* (2023).
- [8] Dongyang Fan, Bettina Messmer, Nikita Doikov, and Martin Jaggi. 2025. On-Device Collaborative Language Modeling via a Mixture of Generalists and Specialists. In *Forty-second International Conference on Machine Learning*.
- [9] William Fedus, Barret Zoph, and Noam Shazeer. 2022. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research* 23, 120 (2022), 1–39.
- [10] Chongyang Gao, Kezhen Chen, et al. 2024. Higher layers need more LoRA experts. *arXiv preprint arXiv:2402.08562* (2024).
- [11] Jingtong Gao, Bo Chen, Xiangyu Zhao, Weiwen Liu, Xiangyang Li, Yichao Wang, Wanyu Wang, Huifeng Guo, and Ruiming Tang. 2025. Lm4rerank: Llm-based auto-reranking framework for recommendations. In *Proceedings of the ACM on Web Conference 2025*. 228–239.
- [12] Avishek Ghosh, Jichan Chung, Dong Yin, and Kannan Ramchandran. 2020. An efficient framework for clustered federated learning. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- [13] Gurobi Optimization, LLC. 2024. Gurobi Optimizer Reference Manual. <https://www.gurobi.com>
- [14] Chaoyang He, Songze Li, Jinhyun So, Mi Zhang, Hongyi Wang, Xiaoyang Wang, Praneeth Vepakomma, Abhishek Singh, Hang Qiu, Li Shen, Peilin Zhao, Yan Kang, Yang Liu, Ramesh Kaskar, Qiang Yang, Murali Annamaram, and Salman Avestimehr. 2020. FedML: A Research Library and Benchmark for Federated Machine Learning. *arXiv preprint arXiv:2007.13518* (2020).
- [15] Edward J Hu, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, Weizhu Chen, et al. [n. d.]. LoRA: Low-Rank Adaptation of Large Language Models. In *International Conference on Learning Representations*.
- [16] Robert A Jacobs, Michael I Jordan, Steven J Nowlan, and Geoffrey E Hinton. 1991. Adaptive mixtures of local experts. *Neural computation* 3, 1 (1991), 79–87.
- [17] Feibo Jiang, Dong Li, et al. 2024. Personalized Wireless Federated Learning for Large Language Models.
- [18] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. 2020. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361* (2020).
- [19] Weirui Kuang, Bingchen Qian, Zitao Li, Daoyuan Chen, Dawei Gao, Xuchen Pan, Yuexiang Xie, Yaliang Li, Bolin Ding, and Jingren Zhou. 2024. Federatedscope-llm: A comprehensive package for fine-tuning large language models in federated learning. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 5260–5271.
- [20] Brian Lester, Rami Al-Rfou, and Noah Constant. 2021. The Power of Scale for Parameter-Efficient Prompt Tuning. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*. 3045–3059.
- [21] Dengchun Li, Yingzi Ma, et al. 2024. MixLoRA: Enhancing large language models fine-tuning with LoRA based mixture of experts. *arXiv preprint arXiv:2404.15159* (2024).
- [22] Tian Li, Anit Kumar Sahu, Manzil Zaheer, Maziar Sanjabi, Ameet Talwalkar, and Virginia Smith. 2020. Federated optimization in heterogeneous networks. In *Proceedings of Machine Learning and Systems (MLSys)*.
- [23] Chin-Yew Lin. 2004. Rouge: A package for automatic evaluation of summaries. In *Text summarization branches out*. 74–81.
- [24] Tao Lin, Lingjing Kong, Sebastian U. Stich, and Martin Jaggi. 2020. Ensemble distillation for robust model fusion in federated learning. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- [25] Xiao Liu, Kaixuan Ji, Yicheng Fu, Weng Tam, Zhengxiao Du, Zhilin Yang, and Jie Tang. 2022. P-Tuning: Prompt Tuning Can Be Comparable to Fine-tuning Across Scales and Tasks. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*. 61–68.
- [26] Peihua Mai, Ran Yan, Zhe Huang, Youjia Yang, and Yan Pang. 2024. Split-and-denoise: Protect large language model inference with local differential privacy. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*. PMLR, 34281–34302.
- [27] Sourab Mangrulkar, Sylvain Gugger, et al. 2022. PEFT: State-of-the-art parameter-efficient fine-tuning methods.
- [28] Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Agüera y Arcas. 2017. Communication-efficient learning of deep networks from decentralized data. In *Artificial intelligence and statistics*. PMLR, 1273–1282.
- [29] Hanzhi Mei, Dongqi Cai, Ao Zhou, Shangguang Wang, and Mengwei Xu. 2024. Fedmoe: Personalized federated learning via heterogeneous mixture of experts. *arXiv preprint arXiv:2408.11304* (2024).
- [30] Liangbo Ning, Ziran Liang, et al. 2025. A survey of webagents: Towards next-generation ai agents for web automation with large foundation models. In *Proceedings of ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 6140–6150.
- [31] Adam Paszke, Sam Gross, et al. 2019. Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems* 32 (2019).
- [32] Daniel Peterson, Pallika Kanani, and Virendra J Marathe. 2019. Private federated learning with domain adaptation. *arXiv preprint arXiv:1912.06733* (2019).
- [33] Jiaxing Qi, Zhongzhi Luan, Shaohan Huang, Carol Fung, Hailong Yang, and Depei Qian. 2024. Fllora: Personalized federated learning of large language model via dual lora tuning. *arXiv preprint arXiv:2406.07925* (2024).
- [34] Tingrui Qiao, Caroline Walker, Chris Cunningham, and Yun Sing Koh. 2025. Thematic-LM: a LLM-based multi-agent system for large-scale thematic analysis. In *Proceedings of the ACM on Web Conference*. 649–658.
- [35] Zhen Qin, Daoyuan Chen, Bingchen Qian, Bolin Ding, Yaliang Li, and Shuangguang Deng. 2024. Federated Full-Parameter Tuning of Billion-Sized Language Models with Communication Cost under 18 Kilobytes. In *International Conference on Machine Learning*. PMLR, 41473–41497.
- [36] Zhen Qin, Shuangguang Deng, Mingyu Zhao, and Xueqiang Yan. 2023. Fedapen: Personalized cross-silo federated learning with adaptability to statistical heterogeneity. In *Proceedings of the 29th ACM SIGKDD conference on knowledge discovery and data mining*. 1954–1964.
- [37] Zhen Qin, Zhaoxin Wu, Bingsheng He, and Shuangguang Deng. 2025. Federated Data-Efficient Instruction Tuning for Large Language Models. In *Findings of the Association for Computational Linguistics, ACL 2025, Vienna, Austria, July 27 - August 1, 2025*. Association for Computational Linguistics, 15550–15568.
- [38] J. Rapin and O. Teytaud. 2018. Nevergrad - a Gradient-Free Optimization Platform.
- [39] Matthias Reisser, Christos Louizos, Efstratios Gavves, and Max Welling. 2021. Federated mixture of experts. *arXiv preprint arXiv:2107.06724* (2021).
- [40] Aarohi Srivastava, Abhinav Rastogi, Abhishek Rao, Abu Awal Md Shoeb, Abubakar Abid, Adam Fisch, et al. 2022. Beyond the Imitation Game: Quantifying and extrapolating the capabilities of language models. *arXiv preprint arXiv:2206.04615* (2022).
- [41] Alysa Ziyang Tan, Han Yu, Lizhen Cui, and Qiang Yang. 2022. Towards personalized federated learning. *IEEE transactions on neural networks and learning systems* 34, 12 (2022), 9587–9603.
- [42] Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang, and Tatsunori B. Hashimoto. 2023. Stanford Alpaca: An Instruction-Following LLaMA Model.
- [43] Hugo Touvron, Thibaut Lavril, et al. 2023. LLaMA: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971* (2023).
- [44] Pablo Villalobos, Anson Ho, Jaime Sevilla, Tamay Besiroglu, Lennart Heim, and Marius Hobbahn. 2024. Position: Will we run out of data? Limits of LLM scaling based on human-generated data. In *Forty-first International Conference on Machine Learning*.
- [45] Alex Wang, Yada Pruksachatkun, Nikita Nangia, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel R. Bowman. 2019. SuperGLUE: A Stickier Benchmark for General-Purpose Language Understanding Systems. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- [46] Lei Wang, Jieming Bian, Letian Zhang, and Jie Xu. 2025. Adaptive LoRA Experts Allocation and Selection for Federated Fine-Tuning. *arXiv preprint arXiv:2509.15087* (2025).
- [47] Yizhong Wang, Swaroop Mishra, et al. 2022. Super-NaturalInstructions: Generalization via Declarative Instructions on 1600+ NLP Tasks. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*. 5085–5109.
- [48] Kang Wei, Jun Li, et al. 2020. Federated learning with differential privacy: Algorithms and performance analysis. *IEEE Transactions on Information Forensics and Security* 15 (2020), 3454–3469.
- [49] Thomas Wolf, Lysandre Debut, et al. 2019. Huggingface's transformers: State-of-the-art natural language processing. *arXiv preprint arXiv:1910.03771* (2019).

- [50] Feijie Wu, Zitao Li, Yaliang Li, Bolin Ding, and Jing Gao. 2024. Fedbiot: Llm local fine-tuning in federated learning without full model. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 3345–3355.
- [51] Mengwei Xu, Dongqi Cai, Yaozong Wu, Xiang Li, and Shangguang Wang. 2024. FwdLLM: Efficient federated finetuning of large language models with perturbed inferences. In *2024 USENIX Annual Technical Conference (ATC)*. 579–596.
- [52] Yiyuan Yang, Guodong Long, Tao Shen, Jing Jiang, and Michael Blumenstein. 2024. Dual-personalizing adapter for federated foundation models. In *Advances in Neural Information Processing Systems (NeurIPS)*. 39409–39433.
- [53] Liping Yi, Han Yu, Chao Ren, Heng Zhang, Gang Wang, Xiaoguang Liu, and Xiaoxiao Li. 2024. pFedMoE: Data-level personalization with mixture of experts for model-heterogeneous personalized federated learning. *arXiv preprint arXiv:2402.01350* (2024).
- [54] Tao Yu, Eugene Bagdasaryan, and Vitaly Shmatikov. 2020. Salvaging federated learning by local adaptation. *arXiv preprint arXiv:2002.04758* (2020).
- [55] Jianyi Zhang, Saeed Vahidian, Martin Kuo, Chunyuan Li, Ruiyi Zhang, Tong Yu, Guoyin Wang, and Yiran Chen. 2024. Towards building the federatedgpt: Federated instruction tuning. In *ICASSP 2024-2024 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 6915–6919.
- [56] Pengyu Zhang, Yingbo Zhou, Ming Hu, Junxian Feng, Jiawen Weng, and Mingsong Chen. 2024. Personalized federated instruction tuning via neural architecture search. *arXiv preprint arXiv:2402.16919* (2024).
- [57] Xiang Zhang, Junbo Zhao, and Yann LeCun. 2015. Character-level Convolutional Networks for Text Classification. In *Proceedings of the International Conference on Learning Representations (ICLR)*.

A More Related Work

A.1 Federated Fine-Tuning for LLMs

Current federated fine-tuning approaches for LLMs primarily focus on improving system efficiency to handle the significant resource demands of these models. To reduce communication overhead, many methods adopt PEFT techniques [4, 55] or employ Zero-Order Optimization (ZOO) to drastically compress model updates [35]. To lower on-device computational and memory footprints, other approaches investigate methods such as quantization [51] or the fine-tuning of compressed LLMs [50]. While these studies have significantly advanced the ecosystem and minimized overheads, they overlook the challenge of data heterogeneity, which causes performance degradation in practical FL applications [41].

A.2 Personalized Federated Learning

Personalized Federated Learning (PFL) has been widely studied to address data heterogeneity. Many approaches focus on parameter-level personalization by adapting model parameters to client-specific distributions through techniques such as fine-tuning [54], regularization [22], clustering [12], and parameter decoupling [5]. Although these methods help mitigate performance degradation, their homogeneous model architectures limit personalization capability. For greater adaptability, some works explore architecture-level personalization using knowledge distillation [24] and ensemble learning [36]. However, knowledge distillation often requires additional data (e.g., public datasets), which contradicts the federated learning goal of addressing data scarcity. Meanwhile, ensemble learning necessitates training multiple models locally—an impractical solution given the high resource cost of training a single LLM.

A.3 Personalized Federated Learning with Mixture of Experts

Mixture-of-Experts (MoE) architectures have attracted attention in PFL due to their ability to achieve model heterogeneity while maintaining constant computational overhead. Early MoE-based PFL research focused on small-scale models like CNNs and RNNs, often

treating the entire model as a single expert. Some works [32, 53] combined a shared expert with a personalized expert using a gating network. Others supported mixing multiple experts through personalized weighting coefficients [39] or client selection [7]. However, these works are primarily designed for small-scale neural networks and incur significant memory and communication overhead, making them unsuitable for LLMs. While a recent effort adopts light-weight experts to improve training efficiency [1], it still restricts personalization to the parameter level.

B Preliminaries

B.1 Mixture of Experts and A Lightweight Improvement

MoE [9, 16] enables the scaling of LLMs with constant computational overhead. A typical MoE layer consists of a linear router and multiple sub-modules termed “experts”. Given a specific input, the router sparsely selects a fixed number of experts based on its features. This process can be formalized as

$$\begin{aligned} p_i &= \text{softmax}(\mathbf{W}_r \mathbf{x})_i \\ \mathcal{T} &= \text{top-}k(p_i) \\ \mathbf{y} &= \sum_{i \in \mathcal{T}} p_i E_i(\mathbf{x}), \end{aligned} \quad (19)$$

where $\mathbf{x} \in \mathbb{R}^d$ represents the input features, $\mathbf{W}_r \in \mathbb{R}^{N \times d}$ is the parameter weights of the router (with N experts), p_i represents the weighting coefficient of expert i for input $\mathbf{x}$, $\mathcal{T}$ is the set of indices of the k selected experts, E_i is the i -th expert sub-module (such as a two-layer feed-forward neural network), and $\mathbf{y}$ is the final output.

The adaptive collaboration of domain experts in MoE enables effective adaptation to diverse data distributions, making it a promising solution for personalizing client models in FL. However, applying MoE to federated LLM fine-tuning incurs significant communication costs due to the transfer of dense expert sub-modules between the server and clients. A natural solution to this limitation is to make MoE experts more lightweight.

Mixture-of-LoRA-Experts (MoLE) architectures [10, 21] address this by using LoRA adapters as experts within the MoE framework.

Low-rank adaptation (LoRA) [15] is a widely adopted Parameter-Efficient Fine-Tuning (PEFT) technique for LLMs. It assumes that the updates $\Delta \mathbf{W}$ to the pre-trained parameters $\mathbf{W} \in \mathbb{R}^{h \times d}$ are highly sparse and can be approximated by the product of two low-rank matrices $\mathbf{A} \in \mathbb{R}^{r \times d}$ and $\mathbf{B} \in \mathbb{R}^{h \times r}$, i.e., LoRA adapters, with $r \ll \min(d, h)$ to minimize parameter size. The fine-tuned parameters $\mathbf{W}$ during forward computation can then be expressed as:

$$\mathbf{y} = \mathbf{W}\mathbf{x} + \Delta \mathbf{W}\mathbf{x} = \mathbf{W}\mathbf{x} + \mathbf{B}\mathbf{A}\mathbf{x}. \quad (20)$$

MoLE substantially reduces the scale of MoE experts with the lightweight nature of LoRA adapters. The forward computation of MoLE is given by:

$$\begin{aligned} p_i &= \text{softmax}(\mathbf{W}_r \mathbf{x})_i \\ \mathcal{T} &= \text{top-}k(p_i) \\ \mathbf{y} &= \mathbf{W}\mathbf{x} + \sum_{i \in \mathcal{T}} p_i \mathbf{B}_i \mathbf{A}_i \mathbf{x}. \end{aligned} \quad (21)$$

B.2 Limitations of Vanilla MoLE for FL

Assigning varying numbers of domain experts to instances of a traditional MoLE module across clients challenges federated aggregation. Consider the following scenario: if two client instances of a MoLE module are assigned to three and five domain experts, respectively, the corresponding router dimensions would be $3 \times d$ and $5 \times d$, as defined in (21). This dimensional inconsistency prevents direct aggregation of routers via FedAvg [28]. The core issue originates that the parameter dimension of a traditional MoLE router is tied to the number of domain experts, resulting in shape mismatches across module instances with varying domain expert counts.

C Loss Function

The loss function used during local fine-tuning is defined as:

$$\mathcal{L}_i(\mathcal{B} \mid \theta_i) = \frac{1}{|\mathcal{B}|} \sum_{\mathbf{x} \in \mathcal{B}} [\mathcal{L}_i(\mathbf{x} \mid \theta_i) + \beta \cdot \mathcal{L}_i^{\text{LB}}(\mathcal{B} \mid \theta_i)], \quad (22)$$

where $\mathcal{B}$ is a mini-batch of text sequences containing T tokens sampled from client i 's training dataset $\mathcal{D}_i^{\text{train}}$, $\mathcal{L}_i$ represents the NLL loss (refer to (3)), and

$$\mathcal{L}_i^{\text{LB}}(\mathcal{B} \mid \theta_i) = \sum_{m \in \mathcal{M}} \left(|\mathcal{E}_{m,i}| \sum_{j \in \mathcal{E}_{m,i}} \bar{f}_{m,i,j} \cdot \bar{p}_{m,i,j} \right) \quad (23)$$

represents the load balance loss [9] weighted by β , which encourages each domain expert to process a comparable number of tokens, ensuring sufficient training. $\bar{f}_{m,i,j}$ and $\bar{p}_{m,i,j}$ are defined as

$$\begin{aligned} \bar{f}_{m,i,j} &= \frac{1}{T} \sum_{\mathbf{x} \in \mathcal{B}} \sum_{\mathbf{x} \in \mathcal{X}} \mathbb{1}\{\arg \max_{j'} p_{m,i,j'}(\mathbf{x}) = j\} \\ \bar{p}_{m,i,j} &= \frac{1}{T} \sum_{\mathbf{x} \in \mathcal{B}} \sum_{\mathbf{x} \in \mathcal{X}} p_{m,i,j}(\mathbf{x}). \end{aligned} \quad (24)$$

D More Experiments

D.1 Implementation Details

Methods are implemented with PyTorch 2.4.0 [31], PEFT [27] and Transformers [49], and performed on a server with NVIDIA RTX 4090 GPU using BF16 precision, unless stated otherwise.

Our Hybrid dataset is a composite of ten datasets for different tasks, including Dolly-15K [6], Alpaca [42], NI [47], SNLI [3], AG News, Yelp-P [57], and four sub-tasks from SuperGLUE [45]. For all scenarios, each client's local data is randomly split into training (80%), validation (10%), and test (10%) sets. Due to computational constraints, we cap all validation sets at 200 samples, and test sets at 200 for SNLI and 50 for the remaining datasets.

Unless otherwise specified, all methods use Adam optimizer with a batch size of 1, a learning rate $\eta = 5e^{-5}$, and a learning rate decay of 0.99 per round. For LoRA-based approaches, we set $r = 8$, $\alpha = 16$, and a dropout rate of 5%, applying LoRA to Q and V in self-attention blocks. For Prompt Tuning, the virtual prompt is initialized using the soft prompt for Dolly-15K, NI, and SNLI, while for Hybrid and BigBench, it is randomly initialized with 20 virtual tokens. For P-Tuning, we set the virtual token count to 20 and use an MLP as the prompt encoder. In FDLORA, the InnerOpt phase synchronizes the shared and personalized adapters every 5 rounds; the OuterOpt phase uses a learning rate of 1.0 and a momentum

of 0.5; the FusionOpt phase uses nevergrad [38] as the black-box optimizer with an L1 regularization weighted by 0.05. For CoMiGS, the total number of experts is kept the same as in this work for a fair comparison, while other parameters follow the recommendations in the corresponding paper[8]. For FedAMoLE, we use SCIP [2] to solve the optimization problem, setting the total assignable experts per module to 30, with $k^e = 2$, $k^c = 2$, $b = 8$, and $\beta = 1e^{-3}$.

D.2 Additional Accuracy Comparison

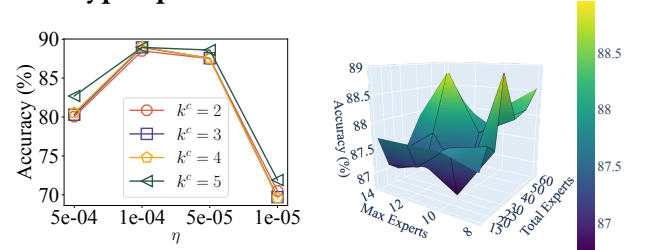
To further validate the effectiveness of FedAMoLE, we compare FedAMoLE to strong baselines with additional LLM sizes (LLaMA-3.2-3B²) and backbones (Qwen-2.5-0.5B³). Results in Table 6 show the consistently superior performance of FedAMoLE.

Table 7: Comparison of accuracy on LLaMA-3.2-1B in a real-world setting. Notation follows Table 1.

FedIT-FT	FDLoRA	FedAMoLE	Gains (%)
45.28±1.00	42.33±1.52	51.28±2.46	13.24

Moreover, we extend the evaluation to three real-world datasets from different domains, i.e., *mtsamples*, *banking77*, and *customer-support-tickets*⁴, each assigned to one of three clients. Results in Table 7 show that FedAMoLE significantly outperforms the best baseline, demonstrating robustness in realistic settings.

D.3 Hyper-parameter Sensitivity



(a) The impact on accuracy by the learning rate η and the number of experts selected by each expert k^c . (b) The impact by the number of total experts and maximum experts selected by each expert b on accuracy.

Figure 9: Hyperparameter sensitivity of FedAMoLE.

Figure 9a presents the hyperparameter sensitivity of FedAMoLE evaluated on SNLI, showing that the performance of FedAMoLE is minimally affected by the number of clients each expert selects. However, over-large or over-small the learning rate declines the performance of FedAMoLE, indicating the need for an appropriate learning rate to ensure optimal performance.

Figure 9b illustrates the impact of the total number of experts per module and the maximum number of experts per client on FedAMoLE's performance. Overall, these two parameters have a limited impact, with model accuracy fluctuating within a narrow 2% range, demonstrating FedAMoLE's robustness to hyperparameter settings. As the figure shows, the model's performance peaks with 30 total experts and $b = 8$. Therefore, we adopt this configuration as the default set of hyperparameters for our experiments.

²Adopt checkpoint from <https://huggingface.co/meta-llama/Llama-3.2-3B>.

³Adopt checkpoint from <https://huggingface.co/Qwen/Qwen2.5-0.5B>.

⁴The Hugging Face repo IDs are harishnair04/mtsamples, PolyAI/banking77, and gorkemsevinc/customer_support_tickets, respectively.

Table 6: Comparison of accuracy on LLaMA-3.2-3B and Qwen-2.5-0.5B. Notation follows Table 1.

Approach	BigBench 1 domain/client	Hybrid 1 task/client	NI 1 task/client	Dolly-15K			SNLI $\alpha = 1.0$	Average
				$\alpha = 0.1$	$\alpha = 1.0$	$\alpha = 100.0$		
LLaMA-3.2-3B	FedIT	54.28±0.72	64.14±0.62	64.80±0.94	33.10±0.54	31.17±1.15	29.72±0.49	52.18
	FedIT-FT	56.22±1.32	65.17±0.40	64.97±1.47	32.35±1.38	31.51±1.11	29.40±0.19	52.83
	FDLoRA	55.42±2.37	66.38±0.54	63.30±1.20	30.82±1.13	29.45±1.59	28.49±1.17	52.05
	FedAMoLE	57.71±0.35	68.55±1.70	66.65±1.06	33.44±1.35	31.61±1.40	30.55±0.38	54.24
	Gains	2.65	3.26	2.58	1.02	0.33	2.77	2.67
Qwen-2.5-0.5B	FedIT	39.69±0.23	60.38±0.32	50.32±1.82	28.82±0.59	28.42±1.72	26.98±0.59	45.41
	FedIT-FT	42.55±0.67	59.88±0.46	53.50±1.31	28.82±0.67	28.04±1.61	26.59±0.55	46.74
	FDLoRA	43.07±1.64	60.82±0.79	51.02±1.40	28.32±1.43	27.94±1.22	27.37±0.91	46.46
	FedAMoLE	49.34±1.93	61.65±0.87	60.31±1.11	29.08±0.97	29.04±1.00	27.54±0.17	49.23
	Gains (%)	14.55	1.38	12.74	0.90	2.18	0.60	5.33

D.4 Stability of Expert Assignment

Table 8: Expert assignment of an HMoLE module on a client in the last 5 rounds. Columns indicate training rounds, rows indicate expert indices, and “✓” denotes assignment.

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30
26																														
27								✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
28								✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
29										✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
30	✓									✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓

Empirical results show that expert assignment under the RSEA strategy remains stable across both runs and training rounds. We illustrate this using experiment logs from the NI dataset (Table 1). Specifically, we randomly select an HMoLE module on a client and track its expert assignments and expert counts over the final five rounds. As shown in Table 8, the assignments converge clearly, with experts 10, 11, 13, 16, 18, and 30 consistently selected and only minor fluctuations observed, demonstrating cross-round stability. Table 9 further shows that the number of assigned experts per round is consistent across seeds, confirming cross-run stability. Note that we omit expert IDs here, as experts are initialized differently under each seed, making ID-level comparisons infeasible.

Table 9: Expert count for the HMoLE module in Table 8 across different random seeds.

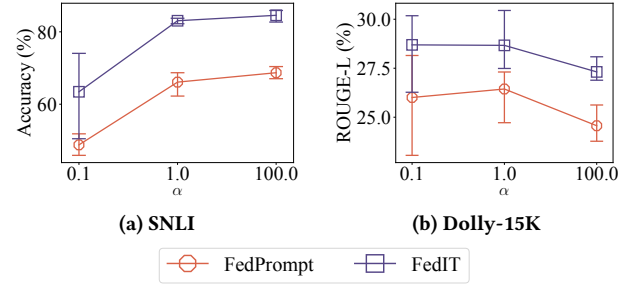
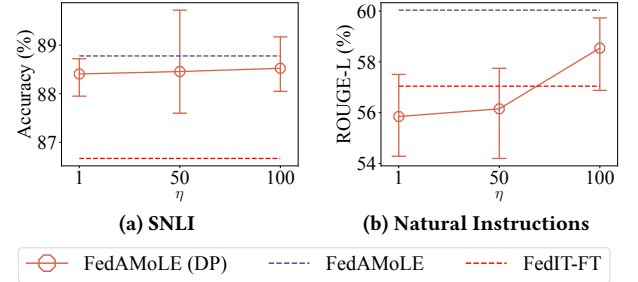
seed	26	27	28	29	30
42	8	8	8	8	8
62	8	8	8	7	8
82	7	7	8	6	8

D.5 Necessity of Personalization

Figure 10 illustrates the performance of two vanilla FL approaches across varying data heterogeneity. On SNLI, performance declines significantly as heterogeneity increases, aligning with existing FL works for relatively small models [41]. Our results confirm that LLM federated fine-tuning is similarly affected, underscoring the need for personalization mechanisms. In contrast, on Dolly-15K, accuracy remains stable despite data heterogeneity, likely because the dataset has fewer task types (8) with smaller inter-task differences, limiting the impact of heterogeneous partitioning.

D.6 Robustness to Differential Privacy

Figure 11 shows the accuracy of FedAMoLE under varying privacy budgets η (§4.5). As expected, applying DP leads to performance degradation, as the added noise in expert and data embeddings

**Figure 10: MTAL of vanilla FL approaches under varying data heterogeneity. Each data point represents the average result over three random seeds, with error bars showing the range between the best and worst values across the three seeds.****Figure 11: MTAL of FedAMoLE under varying privacy budgets η . Each point shows the average result over three seeds, with error bars indicating the min–max range. Dashed lines denote non-private baselines (averaged over three seeds).**

hinders RSEA from accurately measuring expert–client relevance, resulting in suboptimal model architectures. This effect is more pronounced on NI, where greater data heterogeneity makes clients more sensitive to architectural mismatches. Despite the noisy embeddings, FedAMoLE still outperforms the strongest baseline FedIT-FT in most settings, except under high privacy budgets ($\eta = 1, 50$) on NI, confirming its robustness to DP.